

Six Flags Project

David Watson, Matthew Hill, Caleb Short,
Cale Rosenbalm, & Luke Vaughan

Elaboration Phase

Use Case Brainstorming:

Store Inventory - manage pricing, quantity, and location of items

Purchase Item - customer selects and buys items through the system

Validate Item - verify ticket or item is valid before transaction

Update Ride Status - ride operator marks a ride as open, or closed

Schedule Employee - department supervisor assigns shifts to everyone

Hiring - onboard new employees, assigned to departments (Iteration 2)

Security - escort guests out, respond to incidents (Iteration 2)

HR - handle customer complaints, PR incidents (Iteration 2)

Conceptual Class Brainstorming:

Actors:

- Customer, Ride Operator, Ticket Seller, Concessions Worker
- Manager, CEO, Gate Attendant, Security Guard
- Department Supervisor, Inventory Worker

Classes:

- Customer, Item, InventoryItem, Ticket, Concession, Merchandise
- Purchase, Cart, Inventory, ItemStatus
- Ride, RideStatusLog, Location

Interfaces:

- ItemInterface, itemControllerInterface, itemInventoryInterface
- SchedulingSystemInterface, SchedulingControllerInterface
- EmployeeFileHandlerInterface, ScheduleFileHandlerInterface
- ParkInterface, RideInterface

Inception Paragraph:

This system is a management system that supports business operations for the theme park Six Flags. It allows the park to manage its inventory, sales, and employees belonging to different departments within the park. It also allows customers to purchase items like tickets for entry, concessions, and merchandise. The goal of this system is to streamline the park's operations, provide accurate tracking of inventory and sales, and to improve the customer experience.

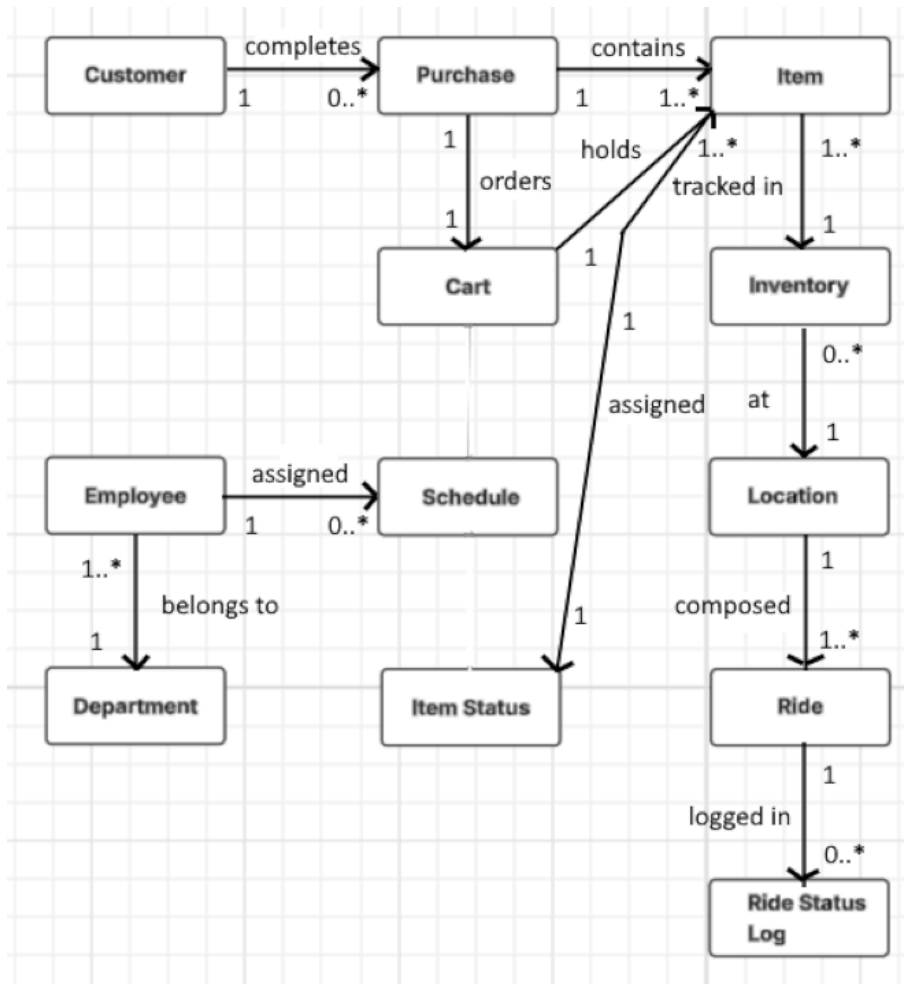
Domain Model:

Coming up with the conceptual classes allows us to create the domain model. We can achieve this by identifying the nouns in the inception paragraph. Nouns such as “system”, “operations”, and “tracking” are thrown away as they are not real business objects. The remaining nouns give us our conceptual classes:

- **Customer** — a person who purchases items at the park
- **Item** — a ticket, concession, or merchandise sold at the park
- **Purchase** — a sales transaction made by a customer

Elaboration

- **Inventory** — tracks the quantity of items available at each location
- **Employee** — a staff member who works at the park
- **Department** — a division of the park (HR, Security, Operations, etc.)
- **Location** — a specific Six Flags park site
- **Ride** — an attraction at a location with an operational status
- **Schedule** — a shift assigned to an employee
- **Item Status** — the validation state of an item (Valid, Invalid, Used)



Iteration One

Iteration 1 - Use Cases

Use Case: Validate Item

Actor: Employee

Precondition:

- Item exists in the system
- Employee has access to validation system
- ID/barcode is on item

Main Success Scenario:

1. Employee scans or enters item into system
2. System retrieves item information
3. System checks if item is valid
4. System verifies item status
5. System confirms item is valid
6. System updates item status (if necessary)
7. Employee proceeds with appropriate action (allow entry, approve transaction)

Alternate Flows:

2a. Item not found

1. System cannot find item in records
2. System displays error message
3. Process is denied

3a. Item invalid

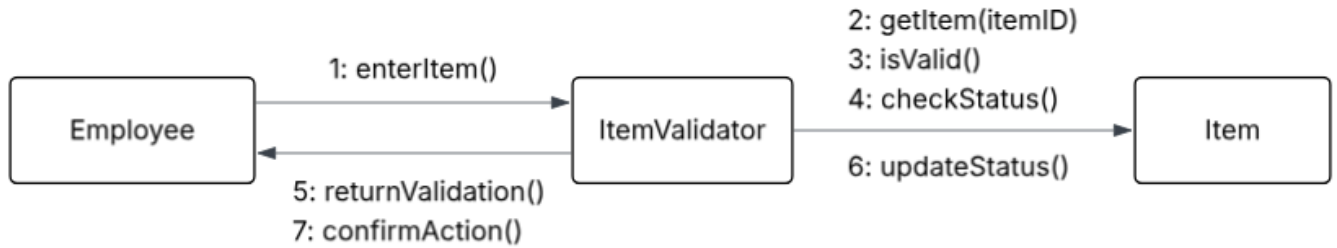
1. System determines item is invalid
2. System displays invalid item message
3. Process is denied

4a. Item already used/processed

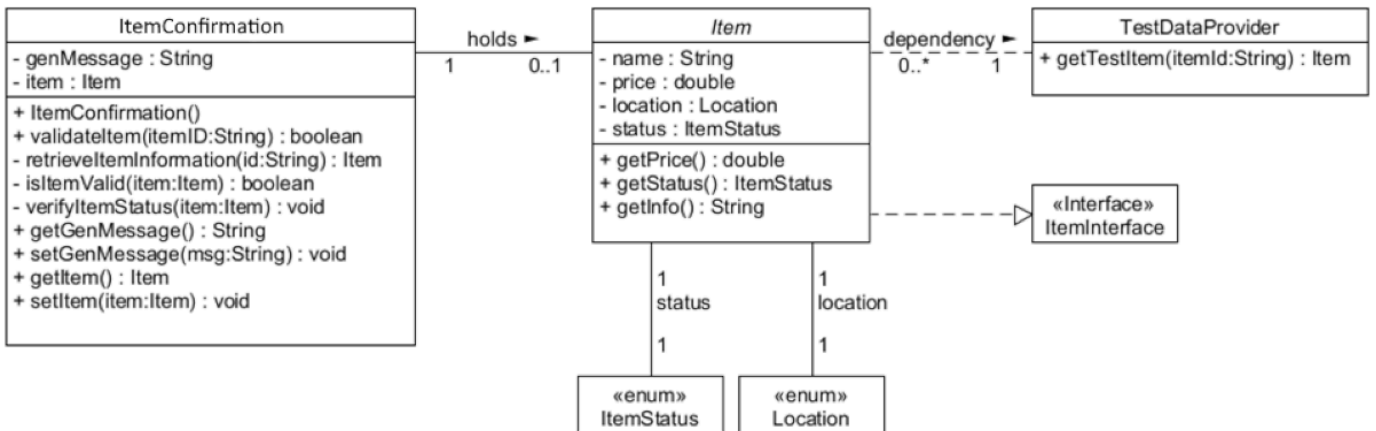
1. System determines item has already been used or processed
2. System displays item status message
3. Process is denied

Iteration 1 - Use Cases

Validate Item Communication Diagram



Validate Item Partial Class Diagram



Iteration 1 - Use Cases

Use Case: Purchase Item

Actor: Customer

Precondition: Online service is active

Main Success Scenario:

1. Customer prompts to buy an item
2. Customer selects desired location
3. Customer selects desired date
4. Customer selects item type (normal vs gold ticket, concessions, etc.)
5. Customer selects item quantity
6. Customer adds item to cart
7. Cart is updated with item information
8. Customer goes to checkout
9. Customer inputs payment information
10. Verify payment information
11. Record purchased item in system
12. Update item inventory with quantity removed
13. Send confirmation to customer

Extensions:

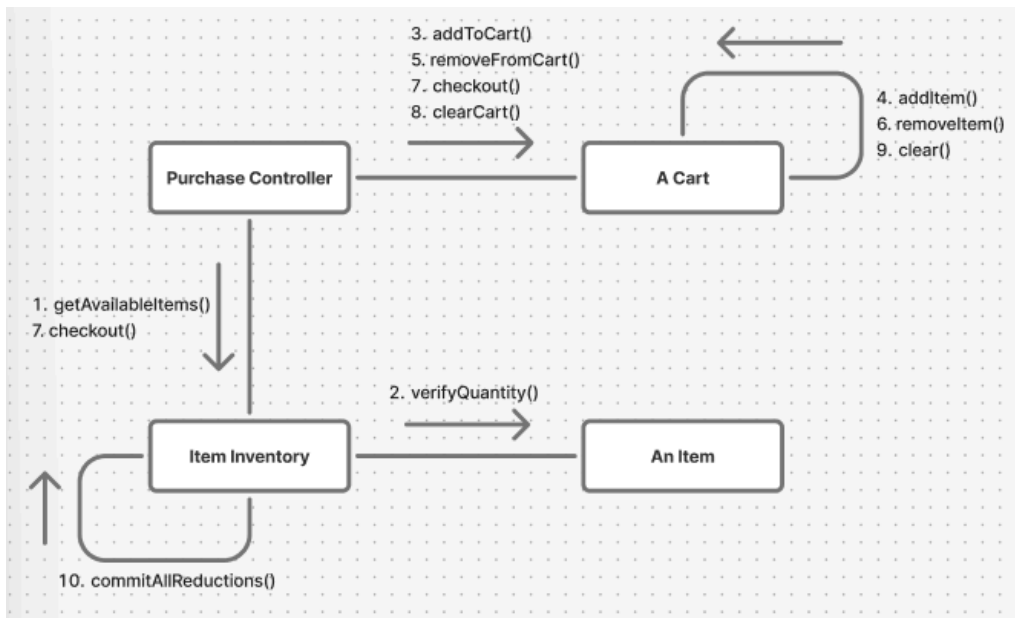
- * Customer cancels transaction
 - Remove all items from cart
 - Cancel sale
- 3a. Location-Date is Unavailable
 - Prompt customer to select a different date
 - Repeat from Step 3
- 4a. Type is fully booked for Location-Date
 - Prompt customer to please select a different type.
 - Repeat from Step 3
- 5a. Quantity exceeds available stock
 - Inform Customer of item limit
 - Continue from Step 5
- 8a. Customer chooses to add more items
 - Repeat from Step 2
- 8b. Customer removes item from cart
 - Remove item from cart
 - Continue from Step 8

Iteration 1 - Use Cases

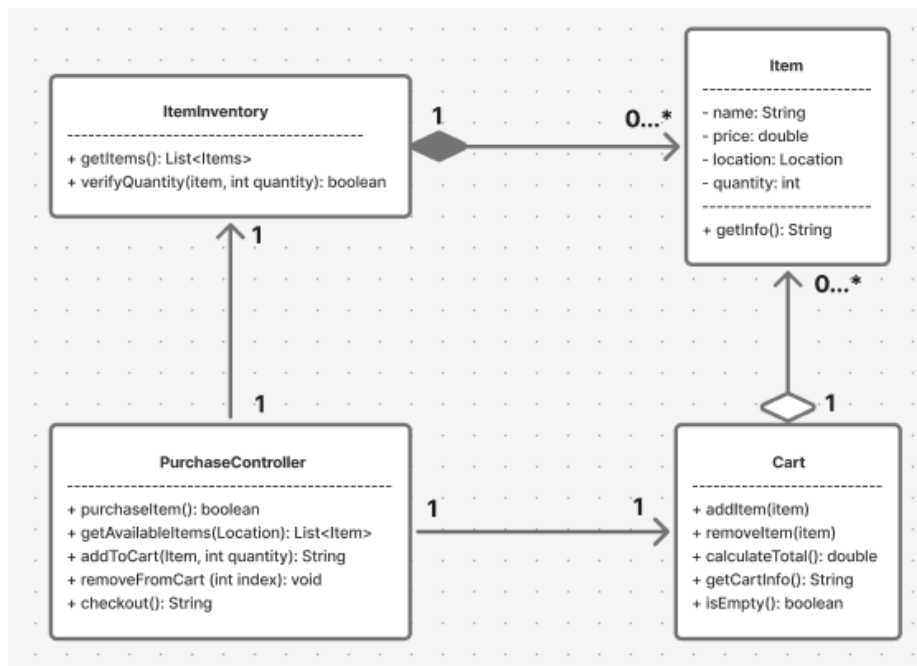
10a. Payment info is incomplete

- Inform customer of required fields
- Repeat from Step 9

Purchase Item Communication Diagram



Purchase Item Partial Class Diagram



Iteration 1 - Use Cases

Use case: Update Ride Status

Actor: Ride Operator

Preconditions:

- Ride OP on duty
- Ride exists in system
- Park is open
- Ride OP is at/assigned with the ride

Main Success Scenario:

1. Ride OP selects a ride from the system
2. System fetches current ride info
3. System displays the current status of the ride
4. Ride OP submits a new status and reason
5. System approves the input
6. System updates the ride status and logs the the changes
7. System confirms the update with the ride OP

Alternate Flows:

3a. Ride not found

1. System cannot find rie in the records
2. System gives an error message
3. Update cancelled

5a. Input Invalid

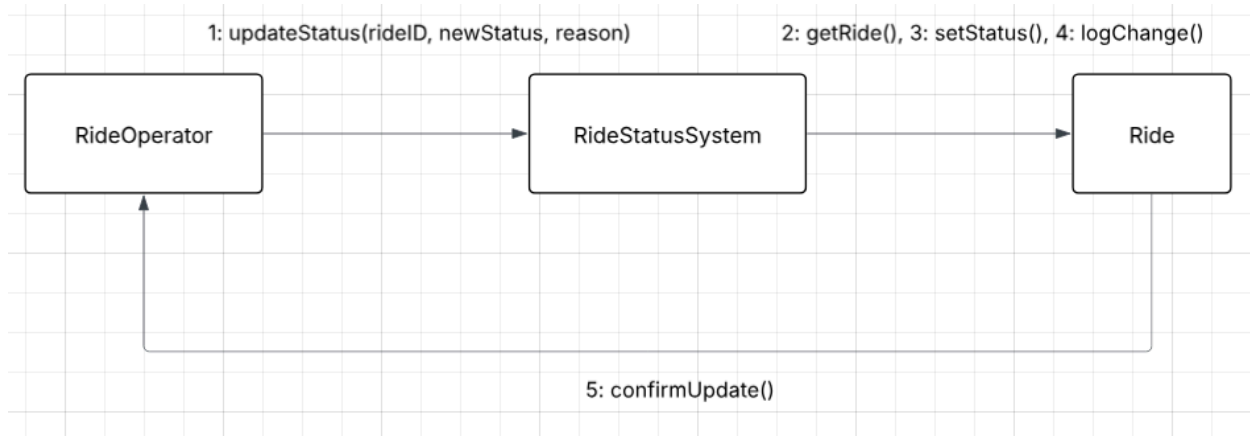
1. System determines input is incomplete or invalid
2. System gives error message
3. Update cancelled

6a. Status already set to selected value

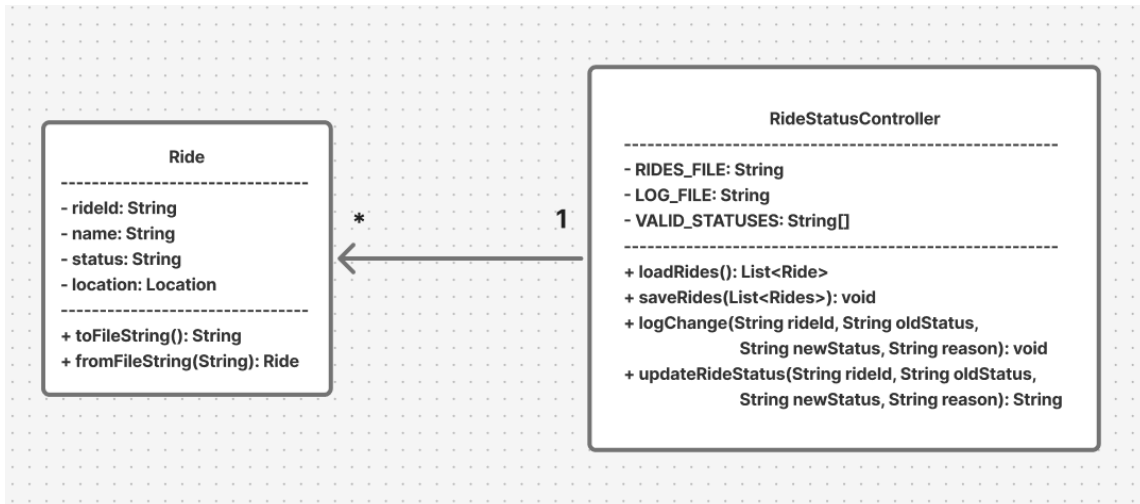
1. System determines no change is needed
2. System displays a "status unchanged" message
3. Update cancelled

Iteration 1 - Use Cases

Update Ride Status Communication Diagram



Update Ride Status Partial Class Diagram



Iteration 1 - Use Cases

Use case: Store Item

Actor: Inventory Worker

Preconditions:

- User is logged into the system
- User has permission to manage inventory
- System is running
- Storage file (inventory file) exists

Main Success Scenario:

1. Inventory Employee selects option to store/add an item
2. System prompts for item details
3. Inventory Employee enters item name
4. Inventory Employee enters item price
5. Inventory Employee enters item quantity
6. Inventory Employee enters item location
7. Inventory Employee submits item information
8. System validates the entered data
9. System creates a new item record
10. System writes item data to inventory file
11. System confirms item has been successfully stored

Alternate Flows:

2a. Inventory Employee cancels operation

1. Inventory Employee cancels the process
2. System stops operation
3. No item is stored

8a. Invalid input data

1. System detects missing or incorrect fields
2. System displays error message
3. Inventory Employee is prompted to re-enter information
4. Resume at step 3

9a. Item already exists

1. System detects duplicate item (same name & location)
2. System prompts manager to update existing item instead
3. Operation is redirected or canceled

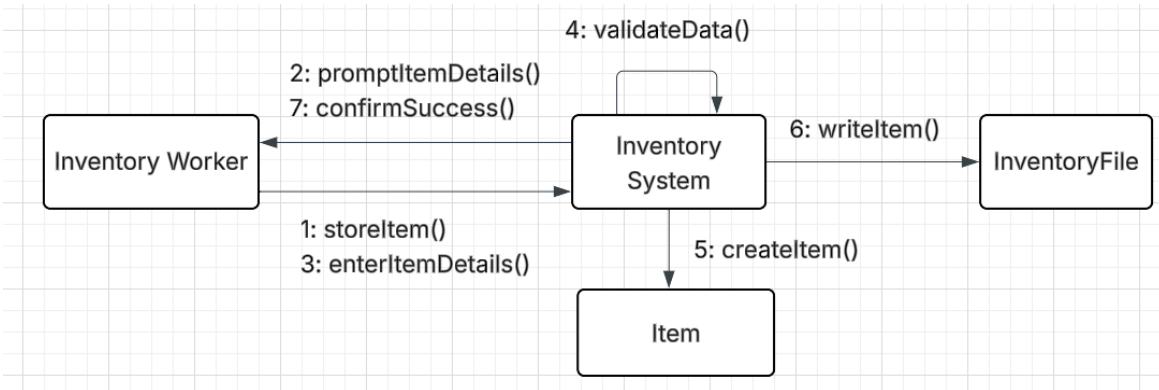
10a. File write error

1. System fails to store item in file

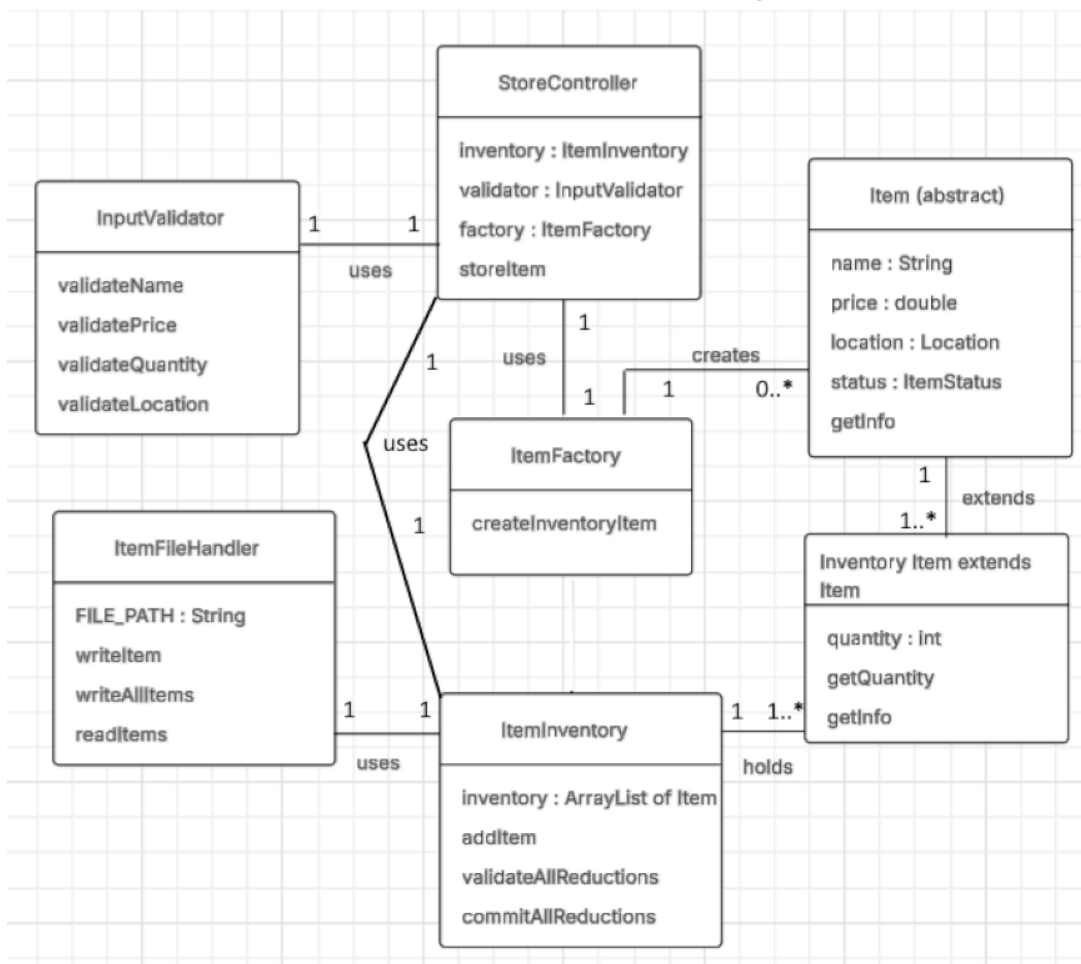
Iteration 1 - Use Cases

2. System displays error message
3. Item is not saved

Store Item Communication Diagram



Store Item Partial Class Diagram



Iteration 1 - Use Cases

Use case: Employee Scheduling

Actor: Department Supervisor

Preconditions:

- Employee file exists
- Schedule file exists
- Employee record exists in the system

Main Success Scenario:

1. Department Supervisor selects option to manage employee schedule
2. System prompts for scheduling details
3. Department supervisor enters employee identifier
4. Department Supervisor submits scheduling information
5. System validates the entered data
6. System checks employee record
7. System checks for scheduling conflicts
8. System process the scheduling information
9. System writes schedule data to schedule file
10. System confirms employee schedule has been successfully managed

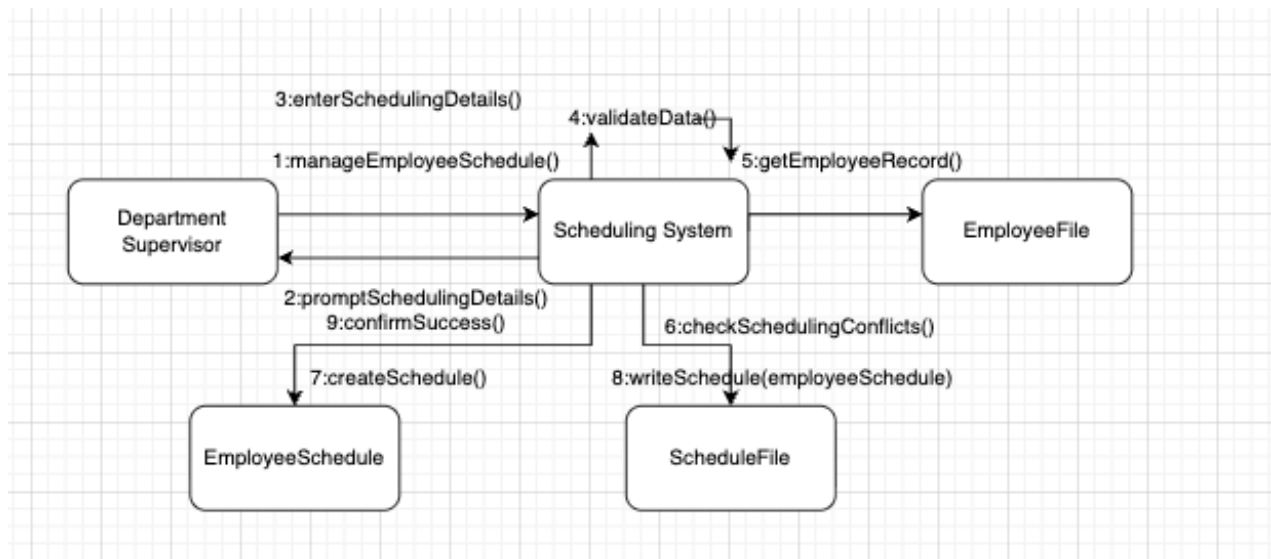
Alternate Flows:

- 1a. Department Supervisor cancels operation
 1. Department Supervisor cancels the process
 2. System stops operation
 3. No schedule information is stored
- 6a. Invalid input data
 1. System detects missing or incorrect scheduling information
 2. System displays error message
 3. Department Supervisor is prompted to re-enter information
 4. Resume at step 3
- 7a. Employee does not exist
 1. System cannot find employee record
 2. System displays error message
 3. Schedule is not created or updated
- 8a. Scheduling conflict exists
 1. System detects conflicting schedule assignment
 2. System displays error message

Iteration 1 - Use Cases

3. Department Supervisor is prompted to revise scheduling information
 4. Resume at step 3
- 9a. File write error
1. System fails to store schedule data in file
 2. System displays error message
 3. Schedule record is not saved

Employee Scheduling Communication Diagram



Employee Scheduling Partial Class Diagram



Iteration 1 - Interfaces

Interface List



Iteration One

Software

Iteration 1 Implementation - Main

```
package main;

import interfaces.ParkInterface;
import ui.UserInterface;

public class Park implements ParkInterface {
    private UserInterface ui;

    public Park() {
        this.ui = new UserInterface();
    }

    @Override
    public void run() {
        ui.start();
    }

    public static void main(String[] args) {
        new Park().run();
    }
}
```

Iteration 1 Implementation - Models

```
package models;

import interfaces.ItemInterface;
import interfaces.ItemStatus;
import interfaces.Location;

public abstract class Item implements ItemInterface {
    private String name;
    private double price;
    private Location location;
    private ItemStatus status;

    public Item(String name, double price, Location location) {
        this.name = name;
        this.price = price;
        this.location = location;
        this.status = ItemStatus.VALID;
    }

    @Override public String getName() { return name; }
    @Override public double getPrice() { return price; }
    @Override public Location getLocation() { return location; }
    @Override public ItemStatus getStatus() { return status; }
    @Override public abstract int getQuantity();

    public void setStatus(ItemStatus status) { this.status = status; }

    @Override
    public String getInfo() {
        return "Name: " + name + " | Location: " + location + " |  
Price: $" + price + " | Status: " + status;
    }
}
```

Iteration 1 Implementation - Models

```
package models;

import interfaces.ItemStatus;
import interfaces.Location;

public class InventoryItem extends Item {
    private int quantity;

    public InventoryItem(String name, double price, Location location,
int quantity) {
        super(name, price, location);
        this.quantity = quantity;
    }

    @Override
    public int getQuantity() { return quantity; }

    @Override
    public String getInfo() {
        return "Name: " + getName() + " | Price: $" + getPrice() + " |
Quantity: " + quantity + " | Location: " + getLocation() + " |
Status: " + getStatus();
    }
}
```

Iteration 1 Implementation - Models

```
package models;

import data.TestDataProvider;
import interfaces.ItemStatus;

public class ItemConfirmation implements
interfaces.ItemConfirmationInterface {
    private String genMessage;
    private Item item;

    private final TestDataProvider dataProvider;

    public ItemConfirmation() {
        this.dataProvider = new TestDataProvider();
        this.genMessage = "";
        this.item = null;
    }

    // Step 1: Employee enters item ID – validate and retrieve
    public boolean validateItem(String itemId) {
        this.item = retrieveItemInformation(itemId);
        if (item == null) {
            setGenMessage("Item not found in records. Process
denied.");
            return false;
        }
        if (!isItemValid(item)) {
            setGenMessage("Item is invalid. Process denied.");
            return false;
        }
        verifyItemStatus(item);
        return item.getStatus() == ItemStatus.VALID;
    }

    // Step 2: Retrieve item from system
    public Item retrieveItemInformation(String id) {
        return dataProvider.getTestItem(id);
    }

    // Step 3: Check if item is valid
    public boolean isItemValid(Item item) {
        return item != null && item.getStatus() != ItemStatus.INVALID;
    }

    // Step 4 & 6: Verify and update item status
```

Iteration 1 Implementation - Models

```
public void verifyItemStatus(Item item) {
    if (item.getStatus() == ItemStatus.USED) {
        setGenMessage("Item has already been used or processed.
Process denied.");
    } else if (item.getStatus() == ItemStatus.VALID) {
        item.setStatus(ItemStatus.USED);
        setGenMessage("Item is valid. Proceed.");
    }
}

public String getGenMessage()          { return genMessage; }
public void setGenMessage(String msg)  { this.genMessage = msg; }
public Item getItem()                  { return item; }
public void setItem(Item item)         { this.item = item; }
public data.TestDataProvider getDataProvider() { return
dataProvider; }
}
```

Iteration 1 Implementation - Models

```
package models;

import java.util.ArrayList;

public class CartManager {
    private ArrayList<Item> cart = new ArrayList<>();

    public void addItem(Item item) {
        cart.add(item);
    }

    public void removeItem(int index) {
        if (index >= 0 && index < cart.size())
            cart.remove(index);
    }

    public void clear() {
        cart.clear();
    }

    public double calculateTotal() {
        double total = 0;
        for (Item item : cart)
            total += item.getPrice() * item.getQuantity();
        return total;
    }

    public String getCartInfo() {
        if (cart.isEmpty()) return "Cart is empty.";
        String output = "Cart:\n";
        for (int i = 0; i < cart.size(); i++) {
            output += " " + (i + 1) + ". " + cart.get(i).getInfo() +
"\n";
        }
        return output;
    }

    public ArrayList<Item> getCart() {
        return cart;
    }

    public boolean isEmpty() {
        return cart.isEmpty();
    }
}
```

Iteration 1 Implementation - Models

```
package models;

public class Ride {
    private String rideID;
    private String name;
    private String status;
    private String location;

    public Ride(String rideID, String name, String status, String
location) {
        this.rideID    = rideID;
        this.name       = name;
        this.status     = status;
        this.location   = location;
    }

    public String getRideID()    { return rideID; }
    public String getName()      { return name; }
    public String getStatus()    { return status; }
    public String getLocation() { return location; }
    public void setStatus(String status) { this.status = status; }

    public String toFileString() {
        return rideID + "," + name + "," + status + "," + location;
    }

    public static Ride fromFileString(String line) {
        String[] parts = line.split(",");
        return new Ride(parts[0], parts[1], parts[2], parts[3]);
    }
}
```

Iteration 1 Implementation - Models

```
package models;

public class Employee {
    private final String employeeId;
    private final String employeeName;
    private final String departmentName;

    public Employee(String employeeId, String employeeName, String
departmentName) {
        this.employeeId      = employeeId;
        this.employeeName    = employeeName;
        this.departmentName = departmentName;
    }

    public String getEmployeeId()      { return employeeId; }
    public String getEmployeeName()    { return employeeName; }
    public String getDepartmentName() { return departmentName; }

    @Override
    public String toString() {
        return "Employee{id='" + employeeId + "', name='" +
employeeName + "', department='" + departmentName + "'}";
    }
}
```

Iteration 1 Implementation - Models

```
package models;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;

public class EmployeeSchedule {
    private static final DateTimeFormatter TIME_FORMATTER =
DateTimeFormatter.ofPattern("HH:mm");

    private final String employeeId;
    private final String employeeName;
    private final String departmentName;
    private final String scheduleDate;
    private final String startTime;
    private final String endTime;

    public EmployeeSchedule(String employeeId, String employeeName,
String departmentName,
                                String scheduleDate, String startTime,
String endTime) {
        this.employeeId      = employeeId;
        this.employeeName    = employeeName;
        this.departmentName  = departmentName;
        this.scheduleDate    = scheduleDate;
        this.startTime       = startTime;
        this.endTime         = endTime;
    }

    public String getEmployeeId()      { return employeeId; }
    public String getEmployeeName()    { return employeeName; }
    public String getDepartmentName() { return departmentName; }
    public String getScheduleDate()    { return scheduleDate; }
    public String getStartTime()       { return startTime; }
    public String getEndTime()         { return endTime; }

    public boolean conflictsWith(String employeeId, String
scheduleDate, String startTime, String endTime) {
        if (!this.employeeId.equals(employeeId) ||
!this.scheduleDate.equals(scheduleDate)) {
            return false;
        }
        try {
            LocalDateTime existingStart = LocalDateTime.parse(this.startTime,
TIME_FORMATTER);
```

Iteration 1 Implementation - Models

```
        LocalDateTime existingEnd    = LocalDateTime.parse(this.endTime,
TIME_FORMATTER);
        LocalDateTime newStart      = LocalDateTime.parse(startTime,
TIME_FORMATTER);
        LocalDateTime newEnd        = LocalDateTime.parse(endTime,
TIME_FORMATTER);
        return newStart.isBefore(existingEnd) &&
newEnd.isAfter(existingStart);
    } catch (DateTimeParseException e) {
        return false;
    }
}
```

Iteration 1 Implementation - Data

```
package data;

import interfaces.ItemInventoryInterface;
import interfaces.Location;
import models.InventoryItem;
import models.Item;

import java.util.ArrayList;
import java.util.List;

public class ItemInventory implements ItemInventoryInterface {
    private ArrayList<Item> inventory;
    private ItemFileHandler fileHandler;

    public ItemInventory() {
        this.fileHandler = new ItemFileHandler();
        this.inventory = fileHandler.readItems();
    }

    @Override
    public List<String> getItems() {
        List<String> names = new ArrayList<>();
        for (Item item : inventory) {
            names.add(item.getInfo());
        }
        return names;
    }

    public ArrayList<Item> getAllItems() {
        return inventory;
    }

    public boolean verifyQuantity(Item item, int quantity) {
        return item.getQuantity() >= quantity;
    }

    public int getQuantityFor(String name, Location location) {
        for (Item item : inventory) {
            if (item.getName().equalsIgnoreCase(name) &&
item.getLocation() == location) {
                return item.getQuantity();
            }
        }
        return 0;
    }
}
```

Iteration 1 Implementation - Data

```
// Returns name of first cart item that can't be fulfilled, or
null if all OK
public String validateAllReductions(ArrayList<Item> cartItems) {
    for (Item cartItem : cartItems) {
        int available = getQuantityFor(cartItem.getName(),
cartItem.getLocation());
        if (available < cartItem.getQuantity()) {
            return cartItem.getName();
        }
    }
    return null;
}

// Call only after validateAllReductions returns null
public void commitAllReductions(ArrayList<Item> cartItems) {
    for (Item cartItem : cartItems) {
        for (int i = 0; i < inventory.size(); i++) {
            Item inv = inventory.get(i);
            if (inv.getName().equalsIgnoreCase(cartItem.getName()))
&&
                inv.getLocation() == cartItem.getLocation()) {
                int newQty = inv.getQuantity() -
cartItem.getQuantity();
                inventory.set(i, new InventoryItem(inv.getName(),
inv.getPrice(), inv.getLocation(), newQty));
                break;
            }
        }
    }
    fileHandler.writeAllItems(inventory);
}

public int increaseQuantity(String name, double price, Location
location, int amount) {
    for (int i = 0; i < inventory.size(); i++) {
        Item item = inventory.get(i);
        if (item.getName().equalsIgnoreCase(name) &&
item.getLocation() == location) {
            int newQuantity = item.getQuantity() + amount;
            inventory.set(i, new InventoryItem(name,
item.getPrice(), location, newQuantity));
            fileHandler.writeAllItems(inventory);
            return newQuantity;
        }
    }
}
```

Iteration 1 Implementation - Data

```
    }
    Item newItem = new InventoryItem(name, price, location,
amount);
    inventory.add(newItem);
    fileHandler.writeItem(newItem);
    return amount;
}

public boolean addItem(Item item) {
    for (Item existing : inventory) {
        if (existing.getName().equalsIgnoreCase(item.getName()) &&
            existing.getLocation() == item.getLocation()) {
            return false;
        }
    }
    inventory.add(item);
    return fileHandler.writeItem(item);
}

public boolean removeItem(Item item) {
    if (inventory.contains(item)) {
        inventory.remove(item);
        return true;
    }
    return false;
}
}
```

Iteration 1 Implementation - Data

```
package data;

import interfaces.Location;
import models.InventoryItem;
import models.Item;

import java.io.*;
import java.util.ArrayList;

public class ItemFileHandler {
    private static final String FILE_PATH = "inventory.txt";

    public boolean writeItem(Item item) {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(FILE_PATH, true))) {
            writer.write(item.getName() + "," +
                item.getPrice() + "," +
                item.getQuantity() + "," +
                item.getLocation().name());
            writer.newLine();
            return true;
        } catch (IOException e) {
            System.out.println("File write error: " + e.getMessage());
            return false;
        }
    }

    public boolean writeAllItems(ArrayList<Item> items) {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(FILE_PATH, false))) {
            for (Item item : items) {
                writer.write(item.getName() + "," +
                    item.getPrice() + "," +
                    item.getQuantity() + "," +
                    item.getLocation().name());
                writer.newLine();
            }
            return true;
        } catch (IOException e) {
            System.out.println("File write error: " + e.getMessage());
            return false;
        }
    }

    public ArrayList<Item> readItems() {
```

Iteration 1 Implementation - Data

```
ArrayList<Item> items = new ArrayList<>();
File file = new File(FILE_PATH);
if (!file.exists()) return items;
try (BufferedReader reader = new BufferedReader(new
FileReader(file))) {
    String line;
    while ((line = reader.readLine()) != null) {
        String[] parts = line.split(",");
        if (parts.length == 4) {
            items.add(new InventoryItem(
                parts[0],
                Double.parseDouble(parts[1]),
                Location.valueOf(parts[3]),
                Integer.parseInt(parts[2])
            ));
        }
    }
} catch (IOException e) {
    System.out.println("File read error: " + e.getMessage());
}
return items;
}
```

Iteration 1 Implementation - Data

```
package data;

import interfaces.EmployeeFileHandlerInterface;
import models.Employee;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.List;

public class EmployeeFileHandler implements
EmployeeFileHandlerInterface {
    private final Path employeeFilePath;

    public EmployeeFileHandler(String employeeFilePath) {
        this.employeeFilePath = Paths.get(employeeFilePath);
    }

    @Override
    public Employee findEmployeeById(String employeeId) throws
IOException {
        for (Employee employee : readAllEmployees()) {
            if (employee.getEmployeeId().equals(employeeId)) {
                return employee;
            }
        }
        return null;
    }

    @Override
    public List<Employee> readAllEmployees() throws IOException {
        List<Employee> employees = new ArrayList<>();
        if (!Files.exists(employeeFilePath)) return employees;

        for (String line : Files.readAllLines(employeeFilePath)) {
            if (line.trim().isEmpty()) continue;
            Employee employee = parseEmployeeRecord(line);
            if (employee != null) employees.add(employee);
        }
        return employees;
    }

    private Employee parseEmployeeRecord(String line) {
```

Iteration 1 Implementation - Data

```
String[] parts = line.split("\\|");
if (parts.length != 3) return null;
return new Employee(parts[0].trim(), parts[1].trim(),
parts[2].trim());
    }
}
```

Iteration 1 Implementation - Data

```
package data;

import interfaces.ScheduleFileHandlerInterface;
import models.EmployeeSchedule;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.List;

public class ScheduleFileHandler implements
ScheduleFileHandlerInterface {
    private final Path scheduleFilePath;

    public ScheduleFileHandler(String scheduleFilePath) {
        this.scheduleFilePath = Paths.get(scheduleFilePath);
    }

    @Override
    public boolean hasConflict(String employeeId, String scheduleDate,
String startTime, String endTime)
        throws IOException {
        for (EmployeeSchedule schedule : readAllSchedules()) {
            if (schedule.conflictsWith(employeeId, scheduleDate,
startTime, endTime)) {
                return true;
            }
        }
        return false;
    }

    @Override
    public void writeSchedule(EmployeeSchedule employeeSchedule)
throws IOException {
        if (scheduleFilePath.getParent() != null) {
            Files.createDirectories(scheduleFilePath.getParent());
        }
        if (Files.notExists(scheduleFilePath)) {
            Files.createFile(scheduleFilePath);
        }
        String record = formatScheduleRecord(employeeSchedule) +
System.lineSeparator();
```

Iteration 1 Implementation - Data

```
Files.writeString(scheduleFilePath, record,
StandardOpenOption.APPEND);
}

@Override
public List<EmployeeSchedule> readAllSchedules() throws
IOException {
    List<EmployeeSchedule> schedules = new ArrayList<>();
    if (Files.notExists(scheduleFilePath)) return schedules;

    for (String line : Files.readAllLines(scheduleFilePath)) {
        if (line.trim().isEmpty()) continue;
        EmployeeSchedule schedule = parseScheduleRecord(line);
        if (schedule != null) schedules.add(schedule);
    }
    return schedules;
}

private EmployeeSchedule parseScheduleRecord(String line) {
    String[] parts = line.split("\\|");
    if (parts.length != 6) return null;
    return new EmployeeSchedule(
        parts[0].trim(), parts[1].trim(), parts[2].trim(),
        parts[3].trim(), parts[4].trim(), parts[5].trim()
    );
}

private String formatScheduleRecord(EmployeeSchedule s) {
    return s.getEmployeeId() + "|" + s.getEmployeeName() + "|" +
s.getDepartmentName() + "|"
        + s.getScheduleDate() + "|" + s.getStartTime() + "|" +
s.getEndTime();
}
}
```

Iteration 1 Implementation - Data

```
package data;

import interfaces.Location;
import models.InventoryItem;
import models.Item;

import java.util.ArrayList;
import java.util.List;

public class TestDataProvider {
    private final List<String> itemIds    = new ArrayList<>();
    private final List<Item>    testItems = new ArrayList<>();

    public TestDataProvider() {
        add("T001", new InventoryItem("General Admission", 39.99,
Location.MAGIC_MOUNTAIN, 100));
        add("T002", new InventoryItem("Gold Pass", 89.99,
Location.MAGIC_MOUNTAIN, 50));
        add("T003", new InventoryItem("Burger Combo", 9.99,
Location.FIESTA_TEXAS, 80));
        add("T004", new InventoryItem("Superman Ride Pass", 15.00,
Location.MAGIC_MOUNTAIN, 60));
        add("T005", new InventoryItem("Iron Rattler Pass", 15.00,
Location.FIESTA_TEXAS, 60));
    }

    private void add(String id, Item item) {
        itemIds.add(id);
        testItems.add(item);
    }

    public Item getTestItem(String itemId) {
        int index = itemIds.indexOf(itemId);
        return index >= 0 ? testItems.get(index) : null;
    }

    public List<String> getAllIds() { return itemIds; }
    public List<Item>  getAllItems() { return testItems; }
}
```

Iteration 1 Implementation - Data

```
package factory;

import interfaces.Location;
import models.InventoryItem;
import models.Item;

public class ItemFactory {
    public Item createInventoryItem(String name, double price,
    Location location, int quantity) {
        return new InventoryItem(name, price, location, quantity);
    }
}
```

Iteration 1 Implementation - Data

```
package validation;

import interfaces.Location;

public class InputValidator {
    public double validatePrice(String input) throws
    IllegalArgumentException {
        double price = Double.parseDouble(input);
        if (price < 0) throw new IllegalArgumentException("Price
cannot be negative.");
        return price;
    }

    public int validateQuantity(String input) throws
    IllegalArgumentException {
        int quantity = Integer.parseInt(input);
        if (quantity <= 0) throw new
    IllegalArgumentException("Quantity must be greater than zero.");
        return quantity;
    }

    public Location validateLocation(String input) throws
    IllegalArgumentException {
        int index = Integer.parseInt(input) - 1;
        if (index < 0 || index >= Location.values().length)
            throw new IllegalArgumentException("Invalid location
selection.");
        return Location.values()[index];
    }

    public String validateName(String input) throws
    IllegalArgumentException {
        if (input == null || input.isBlank())
            throw new IllegalArgumentException("Name cannot be
empty.");
        return input;
    }
}
```

Iteration 1 Implementation - ENUM

```
package interfaces;

public enum Location {
    MAGIC_MOUNTAIN,
    FIESTA_TEXAS,
    GREAT_AMERICA,
    OVER_GEORGIA,
    GREAT_ESCAPE
}
```

```
package interfaces;

public enum ItemStatus {
    VALID,
    INVALID,
    USED
}
```

Iteration 1 Implementation - Controllers

```
package controllers;

import data.ItemInventory;
import interfaces.ItemControllerInterface;
import interfaces.Location;
import models.Item;

import java.util.ArrayList;

public class ItemController implements ItemControllerInterface {
    private PurchaseController purchaseController;
    private StoreController storeController;

    public ItemController() {
        ItemInventory sharedInventory = new ItemInventory();
        this.purchaseController = new
PurchaseController(sharedInventory);
        this.storeController    = new
StoreController(sharedInventory);
    }

    @Override
    public boolean purchaseItem() {
        return true;
    }

    @Override
    public boolean storeItem(String name, String priceInput, String
quantityInput, String locationInput) {
        return storeController.storeItem(name, priceInput,
quantityInput, locationInput) == null;
    }

    public Object[][] getItemTemplates() {
        return storeController.getItemTemplates();
    }

    public ArrayList<Item> getAllItems() {
        return storeController.getAllItems();
    }

    public int getQuantityFor(String name, Location location) {
        return storeController.getQuantityFor(name, location);
    }
}
```

Iteration 1 Implementation - Controllers

```
    public int increaseQuantity(String name, double price, Location
location, int amount) {
        return storeController.increaseQuantity(name, price, location,
amount);
    }

    public String storeItemWithFeedback(String name, String
priceInput, String quantityInput, String locationInput) {
        return storeController.storeItem(name, priceInput,
quantityInput, locationInput);
    }

    public ArrayList<Item> getAvailableItemsByLocation(Location
location) {
        return purchaseController.getAvailableItems(location);
    }

    public String addToCart(Item item, int quantity) {
        return purchaseController.addToCart(item, quantity);
    }

    public void removeFromCart(int index) {
        purchaseController.removeFromCart(index);
    }

    public String checkout() {
        return purchaseController.checkout();
    }

    public double calculateTotal() {
        return purchaseController.calculateTotal();
    }

    public String getCartInfo() {
        return purchaseController.getCartInfo();
    }

    public boolean isCartEmpty() {
        return purchaseController.isCartEmpty();
    }
}
```

Iteration 1 Implementation - Controllers

```
package controllers;

import data.ItemInventory;
import factory.ItemFactory;
import interfaces.Location;
import models.Item;
import validation.InputValidator;

import java.util.ArrayList;

public class StoreController {
    private ItemInventory inventory;
    private InputValidator validator;
    private ItemFactory factory;

    private static final Object[][] ITEM_TEMPLATES = {
        { "General Admission", 39.99 },
        { "Gold Pass", 89.99 },
        { "Burger Combo", 9.99 },
        { "Hot Dog", 5.99 },
        { "Nachos", 6.99 },
        { "Funnel Cake", 7.49 },
        { "Pizza Slice", 5.49 },
        { "Lemonade", 3.99 },
        { "BBQ Sandwich", 8.99 },
        { "Cotton Candy", 3.49 },
        { "Popcorn", 4.49 },
        { "Soft Pretzel", 5.99 },
        { "Superman Ride Pass", 15.00 },
        { "Goliath Ride Pass", 15.00 },
        { "Iron Rattler Pass", 15.00 },
        { "Gold Striker Pass", 15.00 },
        { "Mindbender Pass", 15.00 },
        { "Comet Ride Pass", 15.00 },
    };
};

public StoreController(ItemInventory inventory) {
    this.inventory = inventory;
    this.validator = new InputValidator();
    this.factory = new ItemFactory();
}

public Object[][] getItemTemplates() {
    return ITEM_TEMPLATES;
}
```

Iteration 1 Implementation - Controllers

```
public ArrayList<Item> getAllItems() {
    return inventory.getAllItems();
}

public int getQuantityFor(String name, Location location) {
    return inventory.getQuantityFor(name, location);
}

public int increaseQuantity(String name, double price, Location
location, int amount) {
    return inventory.increaseQuantity(name, price, location,
amount);
}

// Returns null on success, error message on failure
public String storeItem(String name, String priceInput, String
quantityInput, String locationInput) {
    double price;
    int quantity;
    Location location;
    String validatedName;
    try {
        validatedName = validator.validateName(name);
        price          = validator.validatePrice(priceInput);
        quantity       = validator.validateQuantity(quantityInput);
        location        = validator.validateLocation(locationInput);
    } catch (IllegalArgumentException e) {
        return e.getMessage();
    }

    Item item = factory.createInventoryItem(validatedName, price,
location, quantity);
    if (!inventory.addItem(item)) {
        return "Item already exists at this location.";
    }
    return null;
}
}
```

Iteration 1 Implementation - Controllers

```
package controllers;

import data.ItemInventory;
import factory.ItemFactory;
import interfaces.Location;
import models.CartManager;
import models.Item;
import validation.InputValidator;

import java.util.ArrayList;

public class PurchaseController {
    private ItemInventory inventory;
    private CartManager cartManager;
    private InputValidator validator;
    private ItemFactory factory;

    public PurchaseController(ItemInventory inventory) {
        this.inventory = inventory;
        this.cartManager = new CartManager();
        this.validator = new InputValidator();
        this.factory = new ItemFactory();
    }

    public ArrayList<Item> getAvailableItems(Location location) {
        return inventory.getAllItems().stream()
            .filter(i -> i.getLocation() == location &&
i.getQuantity() > 0)

.collect(java.util.stream.Collectors.toCollection(ArrayList::new));
    }

    // Returns null on success, error message on failure
    public String addToCart(Item selectedItem, int quantity) {
        if (!inventory.verifyQuantity(selectedItem, quantity)) {
            return "Insufficient quantity. Only " +
selectedItem.getQuantity() + " available.";
        }
        Item cartItem = factory.createInventoryItem(
            selectedItem.getName(), selectedItem.getPrice(),
selectedItem.getLocation(), quantity);
        cartManager.addItem(cartItem);
        return null;
    }
}
```

Iteration 1 Implementation - Controllers

```
public void removeFromCart(int index) {
    cartManager.removeItem(index);
}

// Returns null on success, error message if transaction aborted
public String checkout() {
    ArrayList<Item> cart = cartManager.getCart();
    String failedItem = inventory.validateAllReductions(cart);
    if (failedItem != null) {
        return "Transaction cancelled. Insufficient stock for: " +
failedItem +
                "\nNo changes were made.";
    }
    inventory.commitAllReductions(cart);
    cartManager.clear();
    return null;
}

public double calculateTotal() {
    return cartManager.calculateTotal();
}

public String getCartInfo() {
    return cartManager.getCartInfo();
}

public boolean isCartEmpty() {
    return cartManager.isEmpty();
}
}
```

Iteration 1 Implementation - Controllers

```
package controllers;

import models.Ride;

import java.io.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class RideStatusController {
    private static final String RIDES_FILE = "data/rides.txt";
    private static final String LOG_FILE =
"data/ride_status_log.txt";

    private static final String[] VALID_STATUSES = {"OPEN", "CLOSED",
"MAINTENANCE"};

    public List<Ride> loadRides() {
        List<Ride> rides = new ArrayList<>();
        File file = new File(RIDES_FILE);
        if (!file.exists()) return rides;
        try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                if (!line.trim().isEmpty())
rides.add(Ride.fromFileString(line));
            }
        } catch (IOException e) {
            System.out.println("Error loading rides: " +
e.getMessage());
        }
        return rides;
    }

    public void saveRides(List<Ride> rides) {
        try (BufferedWriter bw = new BufferedWriter(new
FileWriter(RIDES_FILE))) {
            for (Ride r : rides) { bw.write(r.toFileString());
bw.newLine(); }
        } catch (IOException e) {
            System.out.println("Error saving rides: " +
e.getMessage());
        }
    }
}
```

Iteration 1 Implementation - Controllers

```
    public void logChange(String rideID, String oldStatus, String
newStatus, String reason) {
        try (FileWriter fw = new FileWriter(LOG_FILE, true)) {
            fw.write(rideID + "," + oldStatus + "," + newStatus + "," +
+ reason + "," + LocalDateTime.now() + "\n");
        } catch (IOException e) {
            System.out.println("Error writing log: " +
e.getMessage());
        }
    }

    // Returns null on success, error message on failure
    public String updateRideStatus(String rideID, String newStatus,
String reason) {
        if (newStatus == null || newStatus.isBlank()) return "Status
cannot be empty.";
        if (reason == null || reason.isBlank()) return "Reason
cannot be empty.";

        List<Ride> rides = loadRides();
        for (Ride r : rides) {
            if (r.getRideID().equals(rideID)) {
                if (r.getStatus().equals(newStatus))
                    return "Status is already set to " + newStatus +
". No changes made.";
                String oldStatus = r.getStatus();
                r.setStatus(newStatus);
                saveRides(rides);
                logChange(rideID, oldStatus, newStatus, reason);
                return null; // success
            }
        }
        return "Ride not found in records.";
    }
}
```

Iteration 1 Implementation - Controllers

```
package controllers;

import interfaces.EmployeeFileHandlerInterface;
import interfaces.ScheduleFileHandlerInterface;
import interfaces.SchedulingSystemInterface;
import models.Employee;
import models.EmployeeSchedule;

import java.io.IOException;
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.time.format.ResolverStyle;

public class SchedulingSystem implements SchedulingSystemInterface {
    private static final String SUCCESS_MESSAGE = "Employee
schedule saved successfully.";
    private static final String INVALID_DATA_MESSAGE = "Invalid
scheduling data entered.";
    private static final String EMPLOYEE_NOT_FOUND_MESSAGE = "Employee
does not exist.";
    private static final String CONFLICT_MESSAGE =
"Scheduling conflict exists for this employee.";
    private static final String FILE_ERROR_MESSAGE = "Unable
to save schedule because of a file error.";

    // American date format: MM/dd/yyyy
    private static final DateTimeFormatter DATE_FORMATTER =

DateTimeFormatter.ofPattern("MM/dd/yyyy").withResolverStyle(ResolverS
tyle.STRICT);
    private static final DateTimeFormatter TIME_FORMATTER =

DateTimeFormatter.ofPattern("HH:mm").withResolverStyle(ResolverStyle.
STRICT);

    private final EmployeeFileHandlerInterface employeeFileHandler;
    private final ScheduleFileHandlerInterface scheduleFileHandler;

    public SchedulingSystem(EmployeeFileHandlerInterface
employeeFileHandler,
                           ScheduleFileHandlerInterface
scheduleFileHandler) {
        this.employeeFileHandler = employeeFileHandler;
    }
}
```

Iteration 1 Implementation - Controllers

```
        this.scheduleFileHandler = scheduleFileHandler;
    }

    @Override
    public String processSchedulingRequest(String employeeId, String
scheduleDate, String startTime, String endTime) {
        if (!validateSchedulingData(employeeId, scheduleDate,
startTime, endTime)) {
            return INVALID_DATA_MESSAGE;
        }
        try {
            Employee employee =
employeeFileHandler.findEmployeeById(employeeId);
            if (employee == null) return EMPLOYEE_NOT_FOUND_MESSAGE;

            if (scheduleFileHandler.hasConflict(employeeId,
scheduleDate, startTime, endTime)) {
                return CONFLICT_MESSAGE;
            }

            EmployeeSchedule schedule = new EmployeeSchedule(
                employee.getEmployeeId(), employee.getEmployeeName(),
employee.getDepartmentName(),
                scheduleDate, startTime, endTime);
            scheduleFileHandler.writeSchedule(schedule);
            return SUCCESS_MESSAGE;
        } catch (IOException e) {
            return FILE_ERROR_MESSAGE;
        }
    }

    private boolean validateSchedulingData(String employeeId, String
scheduleDate, String startTime, String endTime) {
        if (isBlank(employeeId) || isBlank(scheduleDate) ||
isBlank(startTime) || isBlank(endTime)) return false;
        try {
            LocalDate.parse(scheduleDate, DATE_FORMATTER);
            LocalTime start = LocalTime.parse(startTime,
TIME_FORMATTER);
            LocalTime end = LocalTime.parse(endTime,
TIME_FORMATTER);
            return start.isBefore(end);
        } catch (DateTimeParseException e) {
            return false;
        }
    }
}
```

Iteration 1 Implementation - Controllers

```
}  
  
private boolean isBlank(String value) {  
    return value == null || value.trim().isEmpty();  
}  
}
```

Iteration 1 Implementation - Controllers

```
package controllers;

import interfaces.SchedulingControllerInterface;
import interfaces.SchedulingSystemInterface;

public class SchedulingController implements
SchedulingControllerInterface {
    private final SchedulingSystemInterface schedulingSystem;

    public SchedulingController(SchedulingSystemInterface
schedulingSystem) {
        this.schedulingSystem = schedulingSystem;
    }

    @Override
    public String manageEmployeeSchedule(String employeeId, String
scheduleDate, String startTime, String endTime) {
        return schedulingSystem.processSchedulingRequest(employeeId,
scheduleDate, startTime, endTime);
    }
}
```

Iteration 1 Implementation - User Interface

```
package ui;

import controllers.ItemController;

import javax.swing.*;
import java.awt.*;

public class UserInterface {
    private JFrame frame;
    private ItemController controller;
    private PurchaseUI purchaseUI;
    private InventoryUI inventoryUI;
    private SchedulingUI schedulingUI;
    private ValidationUI validationUI;
    private RideStatusUI rideStatusUI;

    public UserInterface() {
        this.controller = new ItemController();
        this.frame = new JFrame("Six Flags Management System");
        this.purchaseUI = new PurchaseUI(frame, controller, this);
        this.inventoryUI = new InventoryUI(frame, controller, this);
        this.schedulingUI = new SchedulingUI(frame, this);
        this.validationUI = new ValidationUI(frame, this);
        this.rideStatusUI = new RideStatusUI(frame, this);
    }

    public void start() {
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(500, 450);
        showMainMenu();
        frame.setVisible(true);
    }

    public void showMainMenu() {
        JPanel panel = new JPanel(new GridLayout(6, 1, 10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(30, 80, 30,
80));

        JLabel title = new JLabel("Welcome to Six Flags!",
SwingConstants.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 20));

        JButton purchaseBtn = new JButton("Purchase");
        JButton inventoryBtn = new JButton("Inventory");
        JButton employeeBtn = new JButton("Employee Scheduling");
    }
```

Iteration 1 Implementation - User Interface

```
        JButton validateBtn = new JButton("Validate Item");
        JButton rideBtn      = new JButton("Ride Status");

        purchaseBtn.addActionListener(e ->
purchaseUI.showLocationSelection());
        inventoryBtn.addActionListener(e ->
inventoryUI.showInventoryMenu());
        employeeBtn.addActionListener(e ->
schedulingUI.showSchedulingMenu());
        validateBtn.addActionListener(e ->
validationUI.showValidationMenu());
        rideBtn.addActionListener(e      ->
rideStatusUI.showRideMenu());

        panel.add(title);
        panel.add(purchaseBtn);
        panel.add(inventoryBtn);
        panel.add(employeeBtn);
        panel.add(validateBtn);
        panel.add(rideBtn);

        setPanel(panel);
    }

    public void setPanel(JPanel panel) {
        frame.setContentPane(panel);
        frame.revalidate();
        frame.repaint();
    }

    public void setContentPane(java.awt.Container pane) {
        frame.setContentPane(pane);
        frame.revalidate();
        frame.repaint();
    }
}
```

Iteration 1 Implementation - User Interface

```
package ui;

import controllers.ItemController;
import interfaces.Location;
import models.Item;

import javax.swing.*;
import java.awt.*;
import java.util.ArrayList;

public class InventoryUI {
    private JFrame frame;
    private ItemController controller;
    private UserInterface router;

    private static final String[] LOCATION_NAMES = {
        "Magic Mountain", "Fiesta Texas", "Great America",
        "Over Georgia", "The Great Escape"
    };

    public InventoryUI(JFrame frame, ItemController controller,
        UserInterface router) {
        this.frame = frame;
        this.controller = controller;
        this.router = router;
    }

    public void showInventoryMenu() {
        JPanel panel = new JPanel(new GridLayout(4, 1, 10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(40, 80, 40,
80));

        JLabel label = new JLabel("Inventory Management",
SwingConstants.CENTER);
        label.setFont(new Font("Arial", Font.BOLD, 18));

        JButton addBtn = new JButton("Add Item");
        JButton viewBtn = new JButton("View Inventory");
        JButton backBtn = new JButton("Back");

        addBtn.addActionListener(e -> showLocationSelection());
        viewBtn.addActionListener(e -> showViewInventory());
        backBtn.addActionListener(e -> router.showMainMenu());

        panel.add(label);
```

Iteration 1 Implementation - User Interface

```
        panel.add(addBtn);
        panel.add(viewBtn);
        panel.add(backBtn);

        router.setPanel(panel);
    }

    private void showViewInventory() {
        ArrayList<Item> allItems = controller.getAllItems();

        JPanel outer = new JPanel(new BorderLayout());

        JLabel label = new JLabel("Current Inventory",
SwingConstants.CENTER);
        label.setFont(new Font("Arial", Font.BOLD, 16));
        label.setBorder(BorderFactory.createEmptyBorder(10, 0, 10,
0));
        outer.add(label, BorderLayout.NORTH);

        String[] columns = { "Name", "Price", "Quantity", "Location"
};

        Object[][] data = new Object[allItems.size()][4];
        for (int i = 0; i < allItems.size(); i++) {
            Item item = allItems.get(i);
            data[i][0] = item.getName();
            data[i][1] = String.format("%.2f", item.getPrice());
            data[i][2] = item.getQuantity();
            data[i][3] = item.getLocation().name().replace("_", " ");
        }

        JTable table = new JTable(data, columns) {
            @Override
            public boolean isCellEditable(int row, int col) { return
false; }
        };
        table.setFillViewportHeight(true);
        table.getTableHeader().setReorderingAllowed(false);

        JScrollPane scroll = new JScrollPane(table);
        outer.add(scroll, BorderLayout.CENTER);

        JButton backBtn = new JButton("Back");
        backBtn.addActionListener(e -> showInventoryMenu());
        JPanel south = new JPanel();
        south.add(backBtn);
    }
}
```

Iteration 1 Implementation - User Interface

```
outer.add(south, BorderLayout.SOUTH);

router.setContentPane(outer);
}

private void showLocationSelection() {
    JPanel panel = new JPanel(new GridLayout(7, 1, 10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(20, 60, 20,
60));

    JLabel label = new JLabel("Select a Location:",
SwingConstants.CENTER);
    label.setFont(new Font("Arial", Font.BOLD, 16));
    panel.add(label);

    Location[] locations = Location.values();
    for (int i = 0; i < locations.length; i++) {
        JButton btn = new JButton(LOCATION_NAMES[i]);
        Location loc = locations[i];
        btn.addActionListener(e -> showItemTemplates(loc));
        panel.add(btn);
    }

    JButton backBtn = new JButton("Back");
    backBtn.addActionListener(e -> showInventoryMenu());
    panel.add(backBtn);

    router.setPanel(panel);
}

private void showItemTemplates(Location location) {
    Object[][] templates = controller.getItemTemplates();

    JPanel outer = new JPanel(new BorderLayout());

    JLabel label = new JLabel(
        "Select Item to Add at " + location.name().replace("_", "
") + ":",
        SwingConstants.CENTER);
    label.setFont(new Font("Arial", Font.BOLD, 16));
    label.setBorder(BorderFactory.createEmptyBorder(10, 0, 10,
0));
    outer.add(label, BorderLayout.NORTH);

    JPanel btnPanel = new JPanel(new GridLayout(0, 2, 8, 8));
```

Iteration 1 Implementation - User Interface

```
btnPanel.setBorder(BorderFactory.createEmptyBorder(5, 20, 5,
20));

for (Object[] template : templates) {
    String name      = (String) template[0];
    double price     = (double) template[1];
    int currentQty = controller.getQuantityFor(name,
location);

        JButton btn = new JButton(name + " ($" + price + ") - In
Stock: " + currentQty);
        btn.addActionListener(e -> showQuantityInput(name, price,
location, currentQty));
        btnPanel.add(btn);
    }

    JScrollPane scroll = new JScrollPane(btnPanel);
    scroll.setBorder(null);
    outer.add(scroll, BorderLayout.CENTER);

    JButton backBtn = new JButton("Back");
    backBtn.addActionListener(e -> showLocationSelection());
    JPanel south = new JPanel();
    south.add(backBtn);
    outer.add(south, BorderLayout.SOUTH);

    router.setContentPane(outer);
}

private void showQuantityInput(String name, double price, Location
location, int currentQty) {
    String quantityInput = JOptionPane.showInputDialog(frame,
        name + "\nCurrent stock: " + currentQty + "\n\nEnter
quantity to add:");
    if (quantityInput == null) return;

    int amount;
    try {
        amount = Integer.parseInt(quantityInput.trim());
        if (amount <= 0) {
            JOptionPane.showMessageDialog(frame, "Quantity must be
greater than zero.",
                "Invalid Input", JOptionPane.ERROR_MESSAGE);
            return;
        }
    }
```

Iteration 1 Implementation - User Interface

```
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(frame, "Please enter a valid
number.",
            "Invalid Input", JOptionPane.ERROR_MESSAGE);
        return;
    }

    int newTotal = controller.increaseQuantity(name, price,
location, amount);

    JOptionPane.showMessageDialog(frame,
        "✓ " + name + " updated!\nAdded: " + amount + "\nNew
total stock: " + newTotal,
        "Inventory Updated", JOptionPane.INFORMATION_MESSAGE);

    showItemTemplates(location);
}
}
```

Iteration 1 Implementation - User Interface

```
package ui;

import controllers.ItemController;
import interfaces.Location;
import models.Item;

import javax.swing.*;
import java.awt.*;
import java.util.ArrayList;

public class PurchaseUI {
    private JFrame frame;
    private ItemController controller;
    private UserInterface router;

    private static final String[] LOCATION_NAMES = {
        "Magic Mountain", "Fiesta Texas", "Great America",
        "Over Georgia", "The Great Escape"
    };

    public PurchaseUI(JFrame frame, ItemController controller,
UserInterface router) {
        this.frame      = frame;
        this.controller = controller;
        this.router      = router;
    }

    public void showLocationSelection() {
        JPanel panel = new JPanel(new GridLayout(7, 1, 10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(20, 60, 20,
60));

        JLabel label = new JLabel("Select a Location:",
SwingConstants.CENTER);
        label.setFont(new Font("Arial", Font.BOLD, 16));
        panel.add(label);

        Location[] locations = Location.values();
        for (int i = 0; i < locations.length; i++) {
            JButton btn = new JButton(LOCATION_NAMES[i]);
            Location loc = locations[i];
            btn.addActionListener(e -> showItemSelection(loc));
            panel.add(btn);
        }
    }
}
```

Iteration 1 Implementation - User Interface

```
        JButton backBtn = new JButton("Back");
        backBtn.addActionListener(e -> router.showMainMenu());
        panel.add(backBtn);

        router.setPanel(panel);
    }

    private void showItemSelection(Location location) {
        ArrayList<Item> items =
controller.getAvailableItemsByLocation(location);

        JPanel panel = new JPanel(new GridLayout(items.size() + 2, 1,
10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(20, 60, 20,
60));

        JLabel label = new JLabel("Select Item to Purchase:",
SwingConstants.CENTER);
        label.setFont(new Font("Arial", Font.BOLD, 16));
        panel.add(label);

        if (items.isEmpty()) {
            panel.add(new JLabel("No items available at this
location.", SwingConstants.CENTER));
        } else {
            for (Item item : items) {
                JButton btn = new JButton(item.getName() + " - $" +
item.getPrice() +
                                " (Qty: " +
item.getQuantity() + ")");
                btn.addActionListener(e -> showQuantityInput(item));
                panel.add(btn);
            }
        }

        JButton backBtn = new JButton("Back");
        backBtn.addActionListener(e -> showLocationSelection());
        panel.add(backBtn);

        router.setPanel(panel);
    }

    private void showQuantityInput(Item item) {
        String quantityInput = JOptionPane.showInputDialog(frame,
```

Iteration 1 Implementation - User Interface

```
        "Enter quantity for " + item.getName() + ":\n(Available: "
+ item.getQuantity() + ")");
        if (quantityInput == null) return;

        int quantity;
        try {
            quantity = Integer.parseInt(quantityInput.trim());
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(frame, "Invalid quantity.",
            "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        String error = controller.addToCart(item, quantity);
        if (error != null) {
            JOptionPane.showMessageDialog(frame, error, "Error",
            JOptionPane.ERROR_MESSAGE);
            return;
        }
        showCartMenu();
    }

    public void showCartMenu() {
        JPanel panel = new JPanel(new GridLayout(5, 1, 10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(20, 60, 20,
60));

        JLabel cartLabel = new JLabel("<html>" +
            controller.getCartInfo().replace("\n", "<br>" +
            "</html>",
            SwingConstants.CENTER);
        panel.add(cartLabel);

        JButton continueBtn = new JButton("Continue Shopping");
        JButton removeBtn = new JButton("Remove Item");
        JButton checkoutBtn = new JButton("Checkout");
        JButton backBtn = new JButton("Back to Main");

        continueBtn.addActionListener(e -> showLocationSelection());
        removeBtn.addActionListener(e -> showRemoveItem());
        checkoutBtn.addActionListener(e -> showCheckout());
        backBtn.addActionListener(e -> router.showMainMenu());

        panel.add(continueBtn);
        panel.add(removeBtn);
```

Iteration 1 Implementation - User Interface

```
        panel.add(checkoutBtn);
        panel.add(backBtn);

        router.setPanel(panel);
    }

    private void showRemoveItem() {
        String input = JOptionPane.showInputDialog(frame,
            "Enter item number to remove:\n" +
controller.getCartInfo());
        if (input == null) return;
        try {
            controller.removeFromCart(Integer.parseInt(input) - 1);
            showCartMenu();
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(frame, "Invalid selection.",
"Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    private void showCheckout() {
        String cardInfo = JOptionPane.showInputDialog(frame,
            "Total: $" + String.format("%.2f",
controller.calculateTotal()) + "\nInput Card Info:");
        if (cardInfo == null) return;

        double total = controller.calculateTotal();
        String error = controller.checkout();

        if (error != null) {
            JOptionPane.showMessageDialog(frame, error, "Transaction
Failed", JOptionPane.ERROR_MESSAGE);
        } else {
            JOptionPane.showMessageDialog(frame,
                String.format("Purchase successful!\nTotal charged:
$%.2f\nThank you for your order.", total),
                "Purchase Complete", JOptionPane.INFORMATION_MESSAGE);
            router.showMainMenu();
        }
    }
}
```

Iteration 1 Implementation - User Interface

```
package ui;

import models.Item;
import models.ItemConfirmation;

import javax.swing.*;
import java.awt.*;
import java.util.List;

public class ValidationUI {
    private final JFrame frame;
    private final UserInterface router;
    private final ItemConfirmation itemConfirmation;

    public ValidationUI(JFrame frame, UserInterface router) {
        this.frame = frame;
        this.router = router;
        this.itemConfirmation = new ItemConfirmation();
    }

    public void showValidationMenu() {
        JPanel panel = new JPanel(new GridLayout(3, 1, 10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(40, 80, 40,
80));
        JLabel label = new JLabel("Item Validation",
SwingConstants.CENTER);
        label.setFont(new Font("Arial", Font.BOLD, 18));
        JButton validateBtn = new JButton("Validate Item");
        JButton backBtn = new JButton("Back");
        validateBtn.addActionListener(e -> showItemSelection());
        backBtn.addActionListener(e -> router.showMainMenu());
        panel.add(label); panel.add(validateBtn); panel.add(backBtn);
        router.setPanel(panel);
    }

    private void showItemSelection() {
        List<Item> items =
itemConfirmation.getDataProvider().getAllItems();
        List<String> ids =
itemConfirmation.getDataProvider().getAllIds();

        JPanel panel = new JPanel(new GridLayout(items.size() + 2, 1,
10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(20, 60, 20,
60));
```

Iteration 1 Implementation - User Interface

```
JLabel label = new JLabel("Select Item to Validate:",
SwingConstants.CENTER);
label.setFont(new Font("Arial", Font.BOLD, 16));
panel.add(label);

for (int i = 0; i < items.size(); i++) {
    Item item    = items.get(i);
    String id    = ids.get(i);
    JButton btn = new JButton(item.getName() + " - " +
item.getLocation() + " (" + item.getStatus() + ")");
    btn.addActionListener(e -> validateItem(id));
    panel.add(btn);
}

JButton back = new JButton("Back");
back.addActionListener(e -> showValidationMenu());
panel.add(back);
router.setPanel(panel);
}

private void validateItem(String itemId) {
    boolean ok    = itemConfirmation.validateItem(itemId);
    String result = itemConfirmation.getGenMessage();
    Item item     = itemConfirmation.getItem();
    String display = ok && item != null ? result + "\n\n" +
item.getInfo() : result;
    JOptionPane.showMessageDialog(frame, display, "Validation
Result",
        ok ? JOptionPane.INFORMATION_MESSAGE :
JOptionPane.ERROR_MESSAGE);
    showItemSelection();
}
}
```

Iteration 1 Implementation - User Interface

```
package ui;

import controllers.RideStatusController;
import models.Ride;

import javax.swing.*;
import java.awt.*;
import java.util.List;

public class RideStatusUI {
    private final JFrame frame;
    private final UserInterface router;
    private final RideStatusController controller;

    private static final String[] STATUSES = {"OPEN", "CLOSED",
"MAINTENANCE"};

    public RideStatusUI(JFrame frame, UserInterface router) {
        this.frame      = frame;
        this.router      = router;
        this.controller = new RideStatusController();
    }

    public void showRideMenu() {
        JPanel panel = new JPanel(new GridLayout(3, 1, 10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(40, 80, 40,
80));
        JLabel label = new JLabel("Ride Status Management",
SwingConstants.CENTER);
        label.setFont(new Font("Arial", Font.BOLD, 18));
        JButton updateBtn = new JButton("Update Ride Status");
        JButton backBtn   = new JButton("Back");
        updateBtn.addActionListener(e -> showRideSelection());
        backBtn.addActionListener(e   -> router.showMainMenu());
        panel.add(label); panel.add(updateBtn); panel.add(backBtn);
        router.setPanel(panel);
    }

    private void showRideSelection() {
        List<Ride> rides = controller.loadRides();

        JPanel panel = new JPanel(new GridLayout(rides.size() + 2, 1,
10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(20, 60, 20,
60));
    }
}
```

Iteration 1 Implementation - User Interface

```
JLabel label = new JLabel("Select a Ride:",
SwingConstants.CENTER);
label.setFont(new Font("Arial", Font.BOLD, 16));
panel.add(label);

if (rides.isEmpty()) {
    panel.add(new JLabel("No rides found in data/rides.txt.",
SwingConstants.CENTER));
} else {
    for (Ride ride : rides) {
        JButton btn = new JButton(ride.getName() + " - " +
ride.getLocation() + " [" + ride.getStatus() + "]");
        btn.addActionListener(e -> showUpdateForm(ride));
        panel.add(btn);
    }
}

JButton back = new JButton("Back");
back.addActionListener(e -> showRideMenu());
panel.add(back);
router.setPanel(panel);
}

private void showUpdateForm(Ride ride) {
    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
    panel.setBorder(BorderFactory.createEmptyBorder(20, 40, 20,
40));

    JLabel title = new JLabel("Update: " + ride.getName(),
SwingConstants.CENTER);
    title.setFont(new Font("Arial", Font.BOLD, 16));
    title.setAlignmentX(Component.CENTER_ALIGNMENT);

    JLabel currentLabel = new JLabel("Current Status: " +
ride.getStatus());
    currentLabel.setAlignmentX(Component.CENTER_ALIGNMENT);

    // Status dropdown
    JPanel statusRow = new JPanel(new
FlowLayout(FlowLayout.LEFT));
    statusRow.add(new JLabel("New Status:"));
    JComboBox<String> statusBox = new JComboBox<>(STATUSES);
    statusBox.setSelectedItem(ride.getStatus());
    statusRow.add(statusBox);
}
```

Iteration 1 Implementation - User Interface

```
// Reason input
JPanel reasonRow = new JPanel(new
FlowLayout(FlowLayout.LEFT));
reasonRow.add(new JLabel("Reason:      "));
JTextField reasonField = new JTextField(20);
reasonRow.add(reasonField);

// Buttons
JButton submitBtn = new JButton("Submit");
JButton backBtn   = new JButton("Back");
JPanel btnRow = new JPanel(new FlowLayout(FlowLayout.CENTER,
10, 0));
btnRow.add(backBtn); btnRow.add(submitBtn);

panel.add(title);
panel.add(Box.createVerticalStrut(8));
panel.add(currentLabel);
panel.add(Box.createVerticalStrut(12));
panel.add(statusRow);
panel.add(reasonRow);
panel.add(Box.createVerticalStrut(15));
panel.add(btnRow);

submitBtn.addActionListener(e -> {
    String newStatus = (String) statusBox.getSelectedItem();
    String reason     = reasonField.getText().trim();
    String error      =
controller.updateRideStatus(ride.getRideID(), newStatus, reason);
    if (error != null) {
        JOptionPane.showMessageDialog(frame, error, "Error",
JOptionPane.ERROR_MESSAGE);
    } else {
        JOptionPane.showMessageDialog(frame,
            ride.getName() + " status updated to " + newStatus
+ ".",
            "Update Confirmed",
JOptionPane.INFORMATION_MESSAGE);
        showRideSelection();
    }
});
backBtn.addActionListener(e -> showRideSelection());

router.setPanel(panel);
}
```

Iteration 1 Implementation - User Interface

}

Iteration 1 Implementation - User Interface

```
package ui;

import controllers.SchedulingController;
import controllers.SchedulingSystem;
import data.EmployeeFileHandler;
import data.ScheduleFileHandler;
import interfaces.ScheduleFileHandlerInterface;
import interfaces.SchedulingControllerInterface;
import models.Employee;
import models.EmployeeSchedule;

import javax.swing.*;
import java.awt.*;
import java.util.List;

public class SchedulingUI {
    private final JFrame frame;
    private final UserInterface router;
    private final SchedulingControllerInterface schedulingController;
    private final ScheduleFileHandlerInterface scheduleFileHandler;
    private final EmployeeFileHandler employeeFileHandler;

    public SchedulingUI(JFrame frame, UserInterface router) {
        this.frame = frame;
        this.router = router;
        this.employeeFileHandler = new
EmployeeFileHandler("data/employees.txt");
        this.scheduleFileHandler = new
ScheduleFileHandler("data/schedules.txt");
        this.schedulingController = new SchedulingController(
            new SchedulingSystem(employeeFileHandler,
scheduleFileHandler));
    }

    public void showSchedulingMenu() {
        JPanel panel = new JPanel(new GridLayout(4, 1, 10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(40, 80, 40,
80));
        JLabel label = new JLabel("Employee Management",
SwingConstants.CENTER);
        label.setFont(new Font("Arial", Font.BOLD, 18));
        JButton scheduleBtn = new JButton("Schedule Employee");
        JButton viewBtn = new JButton("View Schedules");
        JButton backBtn = new JButton("Back");
        scheduleBtn.addActionListener(e -> showEmployeeSelection());
    }
}
```

Iteration 1 Implementation - User Interface

```
viewBtn.addActionListener(e -> showViewSchedules());
backBtn.addActionListener(e -> router.showMainMenu());
panel.add(label); panel.add(scheduleBtn); panel.add(viewBtn);
panel.add(backBtn);
router.setPanel(panel);
}

private void showEmployeeSelection() {
    List<Employee> employees;
    try { employees = employeeFileHandler.readAllEmployees(); }
    catch (Exception e) { showError("Could not load employees."); }
return; }

    JPanel panel = new JPanel(new GridLayout(employees.size() + 2,
1, 10, 10));
    panel.setBorder(BorderFactory.createEmptyBorder(20, 60, 20,
60));
    JLabel label = new JLabel("Select Employee:",
SwingConstants.CENTER);
    label.setFont(new Font("Arial", Font.BOLD, 16));
    panel.add(label);
    for (Employee emp : employees) {
        JButton btn = new JButton(emp.getEmployeeName() + " - " +
emp.getDepartmentName());
        btn.addActionListener(e -> showScheduleForm(emp));
        panel.add(btn);
    }
    JButton back = new JButton("Back");
    back.addActionListener(e -> showSchedulingMenu());
    panel.add(back);
    router.setPanel(panel);
}

private void showScheduleForm(Employee employee) {
    // Spinners
    JSpinner month = new JSpinner(new SpinnerNumberModel(1,
12, 1));
    JSpinner day = new JSpinner(new SpinnerNumberModel(1,
31, 1));
    JSpinner year = new JSpinner(new SpinnerNumberModel(2025,
2024, 2030, 1));
    JSpinner sHour = new JSpinner(new SpinnerNumberModel(8,
12, 1));
    JSpinner sMin = new JSpinner(new SpinnerNumberModel(0,
59, 1));
```

Iteration 1 Implementation - User Interface

```
JComboBox<String> sAmPm = new JComboBox<>(new String[]{"AM",
"PM"});
JSpinner eHour = new JSpinner(new SpinnerNumberModel(5, 1,
12, 1));
JSpinner eMin = new JSpinner(new SpinnerNumberModel(0, 0,
59, 1));
JComboBox<String> eAmPm = new JComboBox<>(new String[]{"AM",
"PM"});
eAmPm.setSelectedItem("PM");

// Date row
JPanel dateRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 8,
0));
dateRow.add(new JLabel("Month:")); dateRow.add(month);
dateRow.add(new JLabel("Day:")); dateRow.add(day);
dateRow.add(new JLabel("Year:")); dateRow.add(year);

// Start time row
JPanel startRow = new JPanel(new FlowLayout(FlowLayout.LEFT,
8, 0));
startRow.add(new JLabel("Hour:")); startRow.add(sHour);
startRow.add(new JLabel("Min:")); startRow.add(sMin);
startRow.add(sAmPm);

// End time row
JPanel endRow = new JPanel(new FlowLayout(FlowLayout.LEFT, 8,
0));
endRow.add(new JLabel("Hour:")); endRow.add(eHour);
endRow.add(new JLabel("Min:")); endRow.add(eMin);
endRow.add(eAmPm);

// Button row
JButton submit = new JButton("Submit");
JButton back = new JButton("Back");
JPanel btnRow = new JPanel(new FlowLayout(FlowLayout.CENTER,
10, 0));
btnRow.add(back); btnRow.add(submit);

// Stack everything vertically
JPanel panel = new JPanel();
panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
panel.setBorder(BorderFactory.createEmptyBorder(20, 40, 20,
40));
```

Iteration 1 Implementation - User Interface

```
        panel.add(centered("Scheduling: " +
employee.getEmployeeName(), Font.BOLD, 16));
        panel.add(Box.createVerticalStrut(10));
        panel.add(leftLabel("Date:"));        panel.add(dateRow);
        panel.add(Box.createVerticalStrut(8));
        panel.add(leftLabel("Start Time:")); panel.add(startRow);
        panel.add(Box.createVerticalStrut(8));
        panel.add(leftLabel("End Time:"));   panel.add(endRow);
        panel.add(Box.createVerticalStrut(15));
        panel.add(btnRow);

        submit.addActionListener(e -> {
            String date = String.format("%02d/%02d/%04d",
month.getValue(), day.getValue(), year.getValue());
            String start = toTime((int)sHour.getValue(),
(int)sMin.getValue(), (String)sAmPm.getSelectedItem());
            String end   = toTime((int)eHour.getValue(),
(int)eMin.getValue(), (String)eAmPm.getSelectedItem());
            String result =
schedulingController.manageEmployeeSchedule(employee.getEmployeeId(),
date, start, end);
            boolean ok = result.contains("successfully");
            JOptionPane.showMessageDialog(frame, result, "Result", ok
? JOptionPane.INFORMATION_MESSAGE : JOptionPane.ERROR_MESSAGE);
            if (ok) showSchedulingMenu();
        });
        back.addActionListener(e -> showEmployeeSelection());

        router.setPanel(panel);
    }

    private void showViewSchedules() {
        List<EmployeeSchedule> schedules;
        try { schedules = scheduleFileHandler.readAllSchedules(); }
        catch (Exception e) { showError("Could not load schedules."); }
        return; }

        String[] cols = {"Employee", "Department", "Date", "Start",
"End"};
        Object[][] data = schedules.stream().map(s ->
            new Object[]{s.getEmployeeName(), s.getDepartmentName(),
s.getScheduleDate(), to12Hour(s.getStartTime()),
to12Hour(s.getEndTime())}
        ).toArray(Object[][]::new);
```

Iteration 1 Implementation - User Interface

```
        JTable table = new JTable(data, cols) {
            @Override public boolean isCellEditable(int r, int c) {
return false; }
        };
        JPanel outer = new JPanel(new BorderLayout());
        JLabel lbl = new JLabel("Employee Schedules",
SwingConstants.CENTER);
        lbl.setFont(new Font("Arial", Font.BOLD, 16));
        outer.add(lbl, BorderLayout.NORTH);
        outer.add(new JScrollPane(table), BorderLayout.CENTER);
        JButton back = new JButton("Back");
        back.addActionListener(e -> showSchedulingMenu());
        JPanel south = new JPanel(); south.add(back);
        outer.add(south, BorderLayout.SOUTH);
        router.setContentPane(outer);
    }

    private JLabel centered(String text, int style, int size) {
        JLabel lbl = new JLabel(text, SwingConstants.CENTER);
        lbl.setFont(new Font("Arial", style, size));
        lbl.setAlignmentX(Component.CENTER_ALIGNMENT);
        return lbl;
    }

    private JLabel leftLabel(String text) {
        JLabel lbl = new JLabel(text);
        lbl.setFont(new Font("Arial", Font.BOLD, 13));
        return lbl;
    }

    private String toTime(int hour, int min, String amPm) {
        if (amPm.equals("AM") && hour == 12) hour = 0;
        else if (amPm.equals("PM") && hour != 12) hour += 12;
        return String.format("%02d:%02d", hour, min);
    }

    private String to12Hour(String time24) {
        String[] parts = time24.split(":");
        int hour = Integer.parseInt(parts[0]);
        int min = Integer.parseInt(parts[1]);
        String amPm = hour < 12 ? "AM" : "PM";
        if (hour == 0) hour = 12;
        else if (hour > 12) hour -= 12;
        return String.format("%d:%02d %s", hour, min, amPm);
    }
}
```

Iteration 1 Implementation - User Interface

```
private void showError(String msg) {  
    JOptionPane.showMessageDialog(frame, msg, "Error",  
JOptionPane.ERROR_MESSAGE);  
}  
}
```